

Chapter 3. Asynchrony and Events

We learned that the most important concept in Node is its event-driven model, which is also known as the *non-blocking model*. Node associates asynchronous operations with events and internally takes care of running them independently from other operations. Once an asynchronous operation is done running, Node schedules the execution of your code that depends on that operation.

In this chapter, we'll expand on the important concepts of events and their handler functions as callbacks, promises, and listeners. We'll learn how Node manages the scheduling of asynchronous operations with its event loop and event queues, and see examples of how to create and use custom event emitters and listener functions.

Sync Versus Async Handling

A web browser is a single-threaded environment where all user-related code runs in one main thread. If you have a slow function in your website, users will not be able to even scroll as long as that function is executing. That's because scrolling needs to use the single busy thread.

You get a single thread for your Node code too. Node works with other threads internally, and you can also manually create your own threads when you need to, but it's extremely important that you don't do anything to block your main thread. We saw simple abstract examples of that in [Chapter 1](#). Let's look into an example where blocking the main thread means an entire web server will not be able to serve requests.

If you have a slow synchronous operation in your Node's web server, an incoming HTTP request will block the single precious thread, and your entire web server will not be able to handle any other incoming requests while the slow operation is running under the first incoming request.

To see that in action, let's use the long-running `for` loop to simulate a slow operation within the simple web server example from [Chapter 1](#). To demonstrate the effects of blocking the main thread, let's run the slow operation for only the first incoming request to the server, and skip it for all subsequent incoming requests. We can do that with a simple counter variable:

```
import http from 'node:http';

const slowOperation = () => {
  for (let i = 0; i < 1e9; i++) {
    // Simulate a synchronous delay
  }
};

let counter = 0;

const server = http.createServer((req, res) => {
  counter = counter + 1;
  if (counter === 1) {
    slowOperation();
    res.end('Slow Response');
  } else {
    res.end('Normal Response');
  }
});

server.listen(3000, () => {
  console.log('Server is running...');
});
```

Note how the first request (when `counter` is `1`) will have to wait on the slow operation to complete. The other requests do not need the slow operation and should be fast. However, because the slow operation is synchronous, not only is the first request going to be slow, but all subsequent requests will have to wait on the first one as well!

To test that, run this example and then connect to `http://localhost:3000` multiple times, as shown in [Figure 3-1](#).

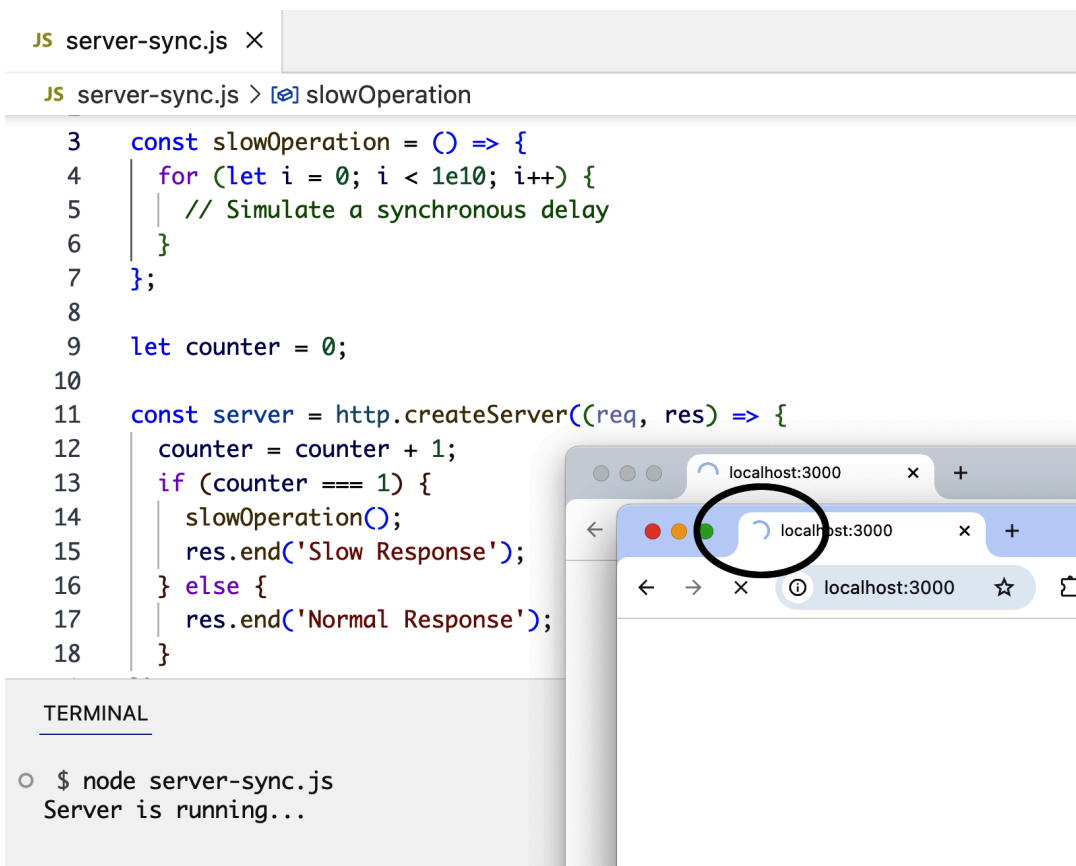


Figure 3-1. A synchronous slow operation

Note how the second browser is spinning, waiting on the first one to be done. Obviously this is very bad, and you should never do something like that.

To simulate executing a slow operation without blocking the main thread, we can use the `setTimeout` function:

```
import http from 'node:http';

const slowOperation = (cb) => {
  // Simulate an asynchronous delay
  setTimeout(
    () => cb(),
    15_000, // delay is in ms
  );
};

let counter = 0;

const server = http.createServer((req, res) => {
  counter = counter + 1;
  if (counter === 1) {
```

```

    slowOperation(() => {
      res.end('Slow Response');
    });
  } else {
    res.end('Normal Response');
  }
});

server.listen(3000, () => {
  console.log('Server is running...');
});

```

The gist of these examples is the same. The first request is delayed because it needs to run after a slow operation. The difference in this example is that the simulated delay is done with an asynchronous API, not a synchronous, long `for` loop.

If you test this code with multiple requests, only the first one will be delayed, but it will not block any other requests from finishing fast, as shown in [Figure 3-2](#).

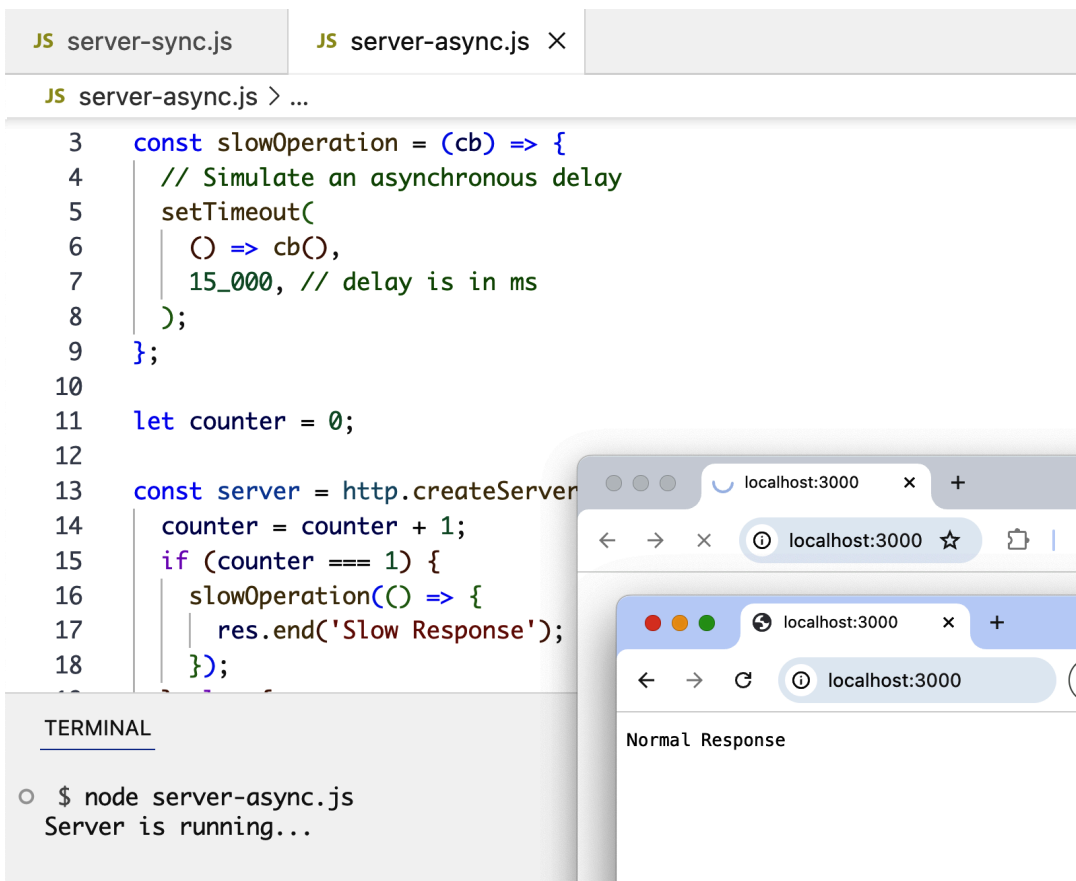


Figure 3-2. An asynchronous slow operation

Note how the second browser in my test responded immediately while the first one was still waiting for the slow operation. This server is basically doing multiple tasks in parallel now, and we did not need to deal with any threads to make it do so.

TIP

There will be cases where your slow operation cannot be made asynchronous with built-in APIs. I'll cover an example of how to handle that without blocking the main thread in [Chapter 7](#).

Handler Functions

Let's review a typical example of an asynchronous callback-style function and how it can be used:

```
// Callback Style

const readFileAsArray = function(file, cb) {
  fs.readFile(file, function(err, data) {
    if (err) {
      return cb(err);
    }
    const lines = data.toString().trim().split('\n');
    cb(null, lines);
  });
};
```

The `readFileAsArray` function takes a file path and a callback function. It reads the file content, splits it into an array of lines, and invokes the callback function after that. It passes the array of lines as an argument to the callback function. If an error occurs while reading the content of the file, the callback function will be invoked with that error as its first argument.

TIP

The `readFileAsArray` function is asynchronous because it uses the asynchronous API method `fs.readFile`. A function becomes asynchronous if it uses any asynchronous functions or if it returns a `Promise` object. We can also make a function asynchronous by using the `async` keyword.

Assume that we have the file *numbers.txt* in the same directory with content like this:

```
10
11
12
13
14
15
```

If we have a task to count the odd numbers in that file, we can use the `readFileAsArray` function to simplify that task:

```
readFileAsArray('./numbers.txt', (err, lines) => {
  if (err) throw err;
  const numbers = lines.map(Number);
  const oddNumbers = numbers.filter(n => n%2 === 1);
  console.log('Odd numbers count:', oddNumbers.length);
});
```

The `readFileAsArray` produces an array of lines in a file. For *numbers.txt*, that would be an array of strings that represent numbers. The callback function in this example parses these strings as numbers and then counts the odd numbers.

Node's callback style is used here. The standard way to structure this style is to have the callback function as the last argument for the asynchronous function. This is one very common convention about Node's callback style. The other convention is how callback functions always have an error-first argument (named `err` in this example). Node's callback style is sometimes referred to as the *error-first* callback style.

Beyond consistency, the error-first callback style conventions are in place for readability and better error handling. If you need to work with Node callback style, you should always structure your functions with these conventions.

If an error happens in the asynchronous function, it'll call the callback function, passing that error down via the `err` argument. If no errors happen, the value of the `err` argument will be `null`. Any callback function written for this style should always check for the presence of that `err` argument before using anything else it has access to (like `lines` in this example). If the error check is not there, errors will just be ignored, and that's bad.

There are a few problems worth mentioning here about callbacks, the most popular of which is what the community named *the pyramid of doom*, or *callback hell*. This happens when you need to do multiple asynchronous operations that depend on each other. You'll have to nest callbacks within each other, making a pyramid-shaped code. This makes it tough to understand the flow of the program.

Another important disadvantage to the callback style is how the callbacks take control of what needs to happen after an asynchronous task is done, and that basically means that we'll have to trust that they'll do the right thing. I explain this in [“An Analogy for Promises”](#).

SYNCHRONOUS CALLBACKS

It's important to understand that the callback pattern by itself does not mean the callback will happen asynchronously. A function can still call its callback synchronously. For example, here's a function that accepts a callback function and may invoke that callback function either synchronously or asynchronously based on a condition:

```
// Example function with both
// synchronous and asynchronous returns

function fileSize (fileName, cb) {
  if (typeof fileName !== 'string') {
    return cb(new TypeError('argument should be string')); // Sync
  }
  fs.stat(fileName, (err, stats) => {
    if (err) { return cb(err); } // Async
    cb(null, stats.size); // Async
  });
}
```

The `cb` function here will be invoked synchronously if `fileSize` is called with a `fileName` that's not a string, and asynchronously otherwise. This is a bad practice because it might lead to unexpected errors. You should design your functions to consume callbacks either always synchronously or always asynchronously. For example, we could use `setTimeout` in the `if` statement to make the `fileSize` consistently work asynchronously.

Promises

Instead of passing a callback function as an argument to an asynchronous function, then handling both success and error cases in the same place, a `Promise` object allows us to handle success and error cases separately, and it also allows us to chain multiple asynchronous calls instead of having to nest them.

If the `readFileAsArray` function supports promises, we can use it as follows:


```

readFileAsArray('./numbers.txt')
  .then(
    (lines) => {
      const numbers = lines.map(Number);
      const oddNumbers = numbers.filter(n => n%2 === 1);
      console.log('Odd numbers count:', oddNumbers.length);
    }
  )
  .catch(
    (err) => console.error(err)
  );

```

We invoke a `.then` function on the return value of the asynchronous function. To support this syntax, the asynchronous function needs to return a `Promise` object. In JavaScript, you can create a `Promise` object using the `Promise` constructor. We'll see how shortly.

In this example, the `.then` function gives us access to the same `lines` array that we get in the callback version, and we can do our processing on it as before. To handle errors, we can chain a `.catch` call, and in there, we get access to the error object if an error happens.

TIP

When working with `Promise` objects using this syntax, you should always add a `.catch` call (just like you should always check the `err` argument in callback functions). When using the `async/await` syntax, we can use the `try/catch` statement to handle any errors from `Promise` objects.

Here's how we can modify the `readFileAsArray` function to support a promise interface, in addition to the callback interface it already supports:

```

// Making a callback function support promises

const readFileAsArray = function (file, cb = () => {}) {
  return new Promise((resolve, reject) => {
    fs.readFile(file, function (err, data) {
      if (err) {
        reject(err);
      }
    });
  });
};

```

```

        return cb(err);
    }
    const lines = data.toString().trim().split('\n');
    resolve(lines);
    cb(null, lines);
  });
});
};

```

We make the function return a `Promise` object (using `new Promise`) that simply wraps the `fs.readFile` async call. Whenever we need to invoke the callback with an error, we invoke the promise `reject` function as well, and whenever we need to invoke the callback with data, we invoke the promise `resolve` function as well.

The only other thing we needed to do in this case is to have a default value for the `cb` argument in case the code is being used with the promise interface. For that, we can simply use a default empty function in the argument: `() => {}`.

Now the `readFileAsArray` function can be used with both the callback pattern and the promise pattern.

While you can still make a pyramid of doom using promises, avoiding it is easy. `Promise` objects support chaining. If you need to do an async operation within the first `.then` handler, you can chain the handler for that operation on the original call instead of nesting it. For example, say that you have a `users` resource and a `posts` resource, and your code needs to start with a `userEmail` value, use that in an async operation to get the `userId` value, then use that in another async operation to get the list of posts from the user. The code would look something like this:

```

getUserIdFromEmail(userEmail)
  .then((userId) => getUserPosts(userId))
  .then((userPosts) => {
    // Do something with userPosts
  })
  .catch((err) => {
    // Do something with err
  });
  .finally(() => {

```

```
// Do something final
// even if the promise is rejected
});
```

Note how we did not need to nest these two operations, and this remains the case no matter how many more asynchronous operations we need. This is certainly more readable than what the callback style could offer for the case.

async/await

The `async/await` pattern improves not only the readability of the code but also its error handling. It also simplifies variable scopes around your asynchronous code.

Here's how we can consume the promisified version of the `readFileAsArray` function with `async/await`:

```
async function countOdd () {
  try {
    const lines = await readFileAsArray('./numbers.txt');
    const numbers = lines.map(Number);
    const oddCount = numbers.filter(n => n%2 === 1).length;
    console.log('Odd numbers count:', oddCount);
  } catch(err) {
    // Do something with err
  }
}

countOdd();
```

We first create an `async` function, which is just a regular function with the word `async` before it. Inside the `async` function, we call the `readFileAsArray` function as if it returns the `lines` variable (although it returns a promise). To make that work, we use the keyword `await` in front of the function call. After that, we continue the code as if the `readFileAsArray` call was synchronous.

To work with errors, we wrap the `async` call in a `try/catch` statement. To get things to run, we execute the `async` function.

The `async/await` syntax simplifies the code, and that is especially true when you need to invoke multiple async functions that depend on each other. Here's an example of how the `async/await` syntax simplifies the code flow of multiple async functions:

```
async function getUserInfo(email) {
  try {
    const userId = await getUserIdFromEmail(email);
    const userPosts = await getUserPosts(userId);

    // Do something with userPosts (and userId)
  } catch (err) {
    // Do something with err
  }
}
```

Besides not having to deal with `.then/.catch` calls and making sure `.then` functions return promises to be chained, there's a subtle but important advantage for the `async/await` version; we can use the results of all the sequential async calls in the same place! We'll need to write more code with the `.then/.catch` version to accomplish that.

TIP

In many cases, you'll need to execute promises in parallel rather than in sequence. One way to do that is with a `Promise.all([promise1, promise2, promise3])` call. You can also use `await [promise1, promise2, promise3]`. The result of both these calls is an array containing the resolved objects in order.

We can use the `async/await` feature with any function that supports a promise interface. We can't use it with callback-style async functions, but as we've seen before, any function designed with the callback style can easily be promisified. In fact, Node has a built-in utility function that can promisify any function that follows the error-first callback style. For example, since the original callback-based `readFileAsArray` function follows the error-first callback style, we can use Node's built-in `node:util` module to promisify it:

```
import { promisify } from 'node:util';

// const readFileAsArray = function(file, cb) { ... }

const readFileAsArrayPromise = promisify(readFileAsArray);

// Use as: await readFileAsArrayPromise(...)
```

Furthermore, some of Node's built-in modules support both the callback style and the promise style for their API. The `node:fs` module is an example. It has a promise-based version of its entire API. Here's a version of `readFileAsArray` written using the `node:fs` promise-based API:

```
import { readFile } from 'node:fs/promises';

const readFileAsArray = async (file) => {
  const data = await readFile(file, 'utf8');
  const lines = data.trim().split('\n');
  return lines;
};
```

TIP

You can use the `await` keyword at the top level of a Node ES module (not within an `async` function). This is useful for a few cases like dynamic imports and resource initializing. However, remember that the `await` keyword pauses the execution of its scope, so using it at the top level will block all the code that comes after it.

Another way to implement handler functions is using event emitter objects, but before we talk about that, let me give a quick analogy about callbacks and promises, and about the control and trust problem I mentioned earlier.

An Analogy for Promises

When you're in a store and you order something that needs time to be prepared, they take your order and name (and money) and tell you to wait to be called when your order is ready. After a while, they call your

name and give you what you ordered. The name you give them is like the callback function that you give to an asynchronous operation. It gets called when the object that was requested is ready.

The callback pattern has many disadvantages. The promise pattern is better. The superiority of promises over callbacks is all about trust and control. Let me explain that with another analogy.

Let's say you're following a recipe to make mansaf, a delicious Levantine dish of rice, lamb, and cooked yogurt. Cooking yogurt requires continuous stirring; if you stop stirring, it burns. This means that while you're stirring the yogurt, you can't do anything else (unless you turn the stove off and cancel the process).

When the yogurt starts boiling, the recipe at that point calls for lowering the heat, adding meat broth, and stirring some more.

If you're the only one cooking, you'll need to do the yogurt-stirring task synchronously. Your body, which is comparable to the single JavaScript thread in this analogy, is blocked for the duration of this synchronous task. You'll have to finish the yogurt cooking before you can start on the rice. This is similar to having synchronous loops in code:

```
putYogurtInPot();
putPotOnStove();

while (yogurtIsNotBoiling) {
  stirYogurt();
  // Now you're blocked until the loop is done
}

lowerHeat();
addMeatBroth(yogurt)

while (yogurtIsNotBoiling) {
  stirYogurt();
  // You're blocked again until the loop is done
}

// Start on the rice
```

If you need to cook both the yogurt and rice simultaneously, you need to get some help. You need to delegate! You need a sous-chef.

Having someone else do the stirring here is like having a module (like `node:fs`) do some async work for you (like reading the content of a file). Just like you use a callback function with an asynchronous call in Node, you need to use a callback function for your sous-chef to give them instructions about their specific tasks (and any ingredients they need):

```
sousChef.handleStirring(rawYogurt, (problem?, boilingYogurt) => {  
  lowerHeat();  
  const yogurtMeatMix = addMeatBroth(boilingYogurt);  
  handleStirring(yogurtMeatMix) // Another async operation  
});  
  
// Start on the rice
```

By doing that, you free your single-threaded body to do something else. You can cook the rice now. You are using an asynchronous API.

So what exactly is the problem here? It's the fact that you lose control of what happens to the yogurt. Not only is the stirring process itself now controlled by your sous-chef, but the tasks that need to be done when the yogurt gets to a boiling point are also controlled by them. There is no guarantee that they will actually perform your instructions exactly like you described them.

Do you trust that they'll correctly identify the boiling point? Will they remember to add meat broth? Will they add enough and not overdo it? Will they remember to lower the heat?

To improve on this process, you can have the sous-chef announce when they think the yogurt is boiling, and then you can confirm and watch them add the meat broth. With these changes, the process becomes a promise-based one, as you basically made your sous-chef promise to announce when the boiling yogurt is ready:

```
try {  
  const boilingYogurt = await sousChef.handleStirringP(rawYogurt);  
  sousChef.lowerHeat();  
}
```

```

    const yogurtMeatMix = sousChef.addMeatBroth(boilingYogurt);
    const cookedYogurt = await sousChef.handleStirringP(yogurtMeatMix);
  } catch(problem) {
    sousChef.reportIt();
  }

  // Start on the rice

```

The only difference between `handleStirring` and `handleStirringP` is that you were promised an outcome for `handleStirringP`, and you can do something with that outcome. You have a little bit of trust and control added to the process. You can add the meat broth yourself (as the blocking part was really only the stirring process):

```

try {
  const boilingYogurt = await sousChef.handleStirringP(rawYogurt);
  you.lowerHeat();
  const yogurtMeatMix = you.addMeatBroth(boilingYogurt);
  const cookedYogurt = await sousChef.handleStirringP(yogurtMeatMix);
} catch(problem) {
  you: inspect(problem) && maybe(retry) || orderSomeTakeout();
}

// Start on the rice

```

More trust does not mean less control or vice versa. This really depends on the nature of the async module you use and its API.

To see the difference in syntax between nesting, chaining, and using `async/await`, let's cook the rice! One way to do so is to soak dry rice in water for a while. You then put the wet rice on the stove, cover it with water, and bring the water to a boil. You'll then need to lower the heat, cover the lid, and steam the rice for a while.

All of these operations are asynchronous. While you're waiting on the rice to soak, you can do something else, and so on. These operations also depend on each other. You can't do them in parallel. You still need to do them in sequence.

Because you start with a dry-rice object and that same object becomes wet rice (and then boiled rice, and then steamed rice), you can compare

this process to working with promises:

```
// Nesting
dryRice.soak()
  .then(wetRice =>
    wetRice.boil()
      .then(boiledRice =>
        boiledRice.steam()
          .then(steamedRice => serve(steamedRice)));
  );

// Chaining
dryRice.soak()
  .then(wetRice => wetRice.boil())
  .then(boiledRice => boiledRice.steam())
  .then(steamedRice => serve(steamedRice));

// async/await
const wetRice = await dryRice.soak();
const boiledRice = await wetRice.boil();
const steamedRice = await boiledRice.steam();

serve(steamedRice);
```

The Event Loop

How exactly does a handler function get executed in the main thread after its asynchronous operation is done outside the main thread?

More importantly, when there are multiple asynchronous operations and multiple handler functions, how are they managed, and in what order do they get back to the main thread?

To answer these questions, we need to understand three main structures: the *call stack*, *event queues*, and the *event loop*.

V8 manages the execution of JavaScript functions using a simple stack structure known as the call stack. A stack is a *last-in, first-out* (LIFO) data structure where the last entity pushed to it is the first entity to be processed.

Every time a function is called, a reference to it gets placed on top of the call stack. When a stacked function calls other functions (including themselves recursively), a reference to each nested function gets placed on top of the call stack.

When a function call is completed, its reference gets popped out of the call stack. Nested function calls get popped out one at a time (from the top of the call stack) to complete the initial call of the first function that was stacked.

Any JavaScript code you write in Node has to be placed in the call stack for V8 to execute it. The call stack is single-threaded, which means when there are functions in the call stack, everything else (including handler functions for asynchronous operations) will have to wait until the call stack is available again. That's why it's important to never write code that will keep the call stack busy (like a long-running `for` loop). As long as the call stack is busy, all your asynchronous operation handlers will be waiting. Any code that needs to run for a long time should be executed outside of the main thread and its one call stack.

While stacking functions in the call stack, when Node gets to an asynchronous task, it bypasses the call stack and processes the task internally. Depending on the nature of the task, Node might use a different thread or utilize the underlying OS asynchronous features.

TIP

Node uses the asynchronous features of its underlying OS to perform non-blocking I/O operations. For CPU-bound tasks and other tasks that can't be handled asynchronously by the OS, Node provides a thread pool via its `libuv` library. This library is the backbone of everything asynchronous in Node. It's where the event loop is implemented, and it provides abstractions of platform-specific details so that Node can run consistently on different operating systems.

When an asynchronous task is completed, Node places its associated handler function in a queue structure known as the event queue (or task queue). A queue is a *first-in, first-out* (FIFO) structure where the first thing added is the first one processed.

For a handler function to run, it needs to be placed on the call stack. That's the job of the event loop (see [Figure 3-3](#)). It's a simple infinite loop with a simple job: when the event queue has handler functions and the call stack is empty, it picks the first queued function in the event queue and puts it on the call stack to have V8 execute it. Then it waits until the call stack is empty again to process the next queued handler function and keeps repeating that until there are no more handler functions in the event queue to process.

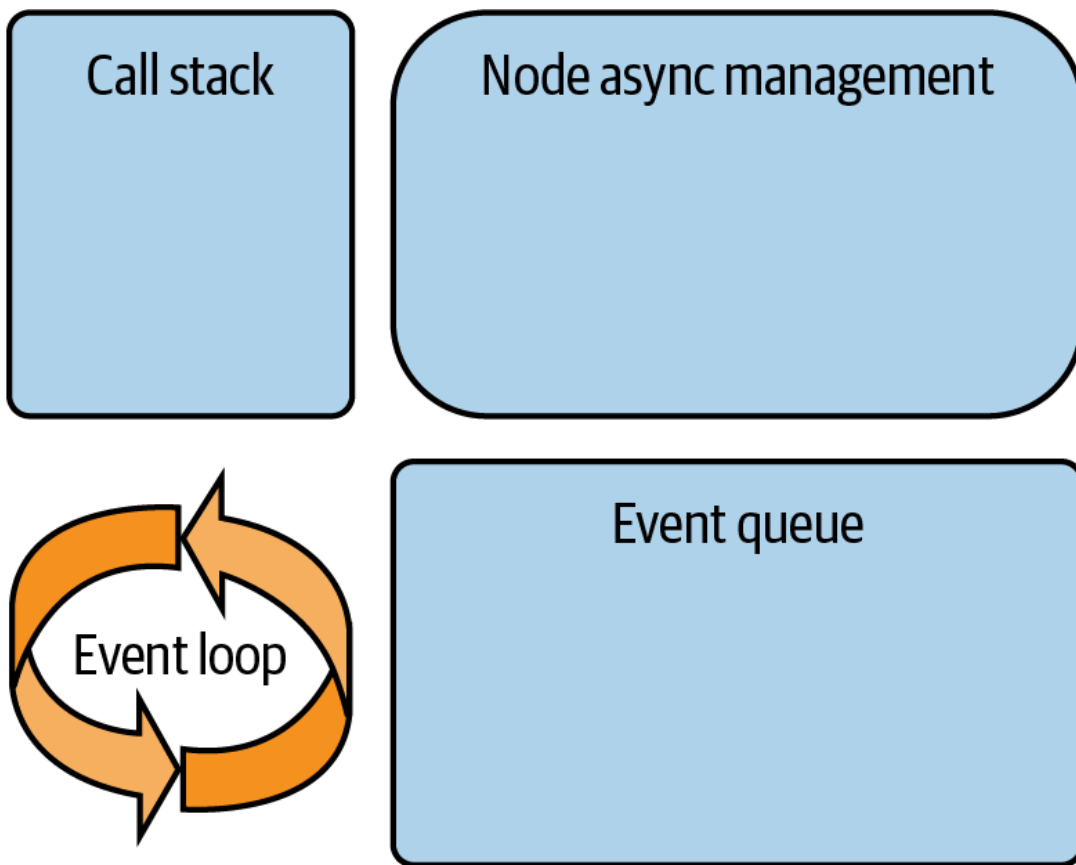


Figure 3-3. Async management and the event loop

This flow happens for any asynchronous operation, whether it's handling an incoming request to a web server, reading a file from the filesystem, starting a timer, or encrypting data. Every asynchronous task has a handler function that gets queued in an event queue, picked up by the event loop, pushed on to the call stack, then executed by V8.

TIP

The event loop has many phases with different priorities, and it works with multiple event queues associated with these phases. There is a phase for timers, another for system-level callbacks, one for I/O operations, and a few more. Promises and high priority tasks are handled in their own queue known as the *microtask queue*.

Another option to organize and use asynchronous code in Node is through the use of event emitter objects and event listener functions. Let's talk about that next.

Event Emitters

Using events to handle asynchronous tasks takes the game to a completely different level. Events are all about good communication. No matter how complex a situation is, good communication makes things a lot better. This is true in life, and it's true with Node events.

The concept in Node is simple: emitter objects emit named events that cause previously registered handler functions to be called.

An emitter object basically has two main features:

- Emitting named events
- Registering (and unregistering) listener functions

To create an event emitter object, we can instantiate from either the `EventEmitter` class or any class that extends it. The latter allows for custom logic that can be shared between multiple event emitter objects:

```
import { EventEmitter } from 'node:events';

class MyEmitter extends EventEmitter {
  // Custom logic for all event emitter objects
}

const myEmitter = new MyEmitter();
```

At any point in the lifecycle of the emitter objects, we can use the `emit` function to emit any named event we want:

```
myEmitter.emit('something-happened', data);

// Or within the MyEmitter class:
this.emit('something-happened', data);
```

We can optionally include data objects after the event-name argument. You can use as many more arguments there as needed, and all of them will be included with the emitted event and made available to the registered handler functions.

We can add handler functions using the `on` method, and they will be executed every time the emitter object emits their associated name event:

```
myEmitter.on('something-happened', (data) => {
  // Do something with data
})
```

To see events in action, here's the `readFileAsArray` example implemented using events instead of callbacks or promises:

```
import fs from 'node:fs';
import { EventEmitter } from 'node:events';

class ReaderEmitter extends EventEmitter {
  readFileAsArray(file) {
    fs.readFile(file, (err, data) => {
      if (err) {
        this.emit('error', err);
        return;
      }
      const lines = data.toString().trim().split('\n');
      this.emit('data', lines);
    });
  }
}

const reader = new ReaderEmitter();
```

```

reader.on('data', (lines) => {
  const numbers = lines.map(Number);
  const oddNumbers = numbers.filter((n) => n % 2 === 1);
  console.log('Odd numbers count: ', oddNumbers.length);
});

reader.on('error', (err) => {
  throw err;
});

reader.readFileAsArray('./numbers.txt');

```

While this is a simple example, you can still see how processing the data gets its own function, and so does the error handling. This is more modular and easier to manage. As the code gets more complex, the event-based approach improves the reusability and scalability of the code. We can create multiple emitters, add more events, and handle each of the events multiple times. In addition to that, the event-based approach simplifies writing focused unit tests for the code, and it makes debugging the code easier as well.

Asynchrony

Just like with the callbacks style, having an emitter object does not mean that events are triggered asynchronously. Let's take a look at an example:

```

import { EventEmitter } from 'node:events';

class WithLog extends EventEmitter {
  execute(taskFunc) {
    this.emit('begin');
    taskFunc();
    this.emit('end');
  }
}

const withLog = new WithLog();

withLog.on('begin', () => console.log('About to execute'));
withLog.on('end', () => console.log('Done with execute'));

withLog.execute(() => console.log('*** Executing task ***'));

```

The class `WithLog` defines one instance function, `execute`. This `execute` function receives one argument, a task function, and emits events before and after its execution.

To see the sequence of what will happen here, we define listeners for both named events and execute a sample task to trigger the events. Here's the output of that:

```
About to execute
*** Executing task ***
Done with execute
```

What I want you to notice about this flow is that it all happens synchronously. There is nothing asynchronous about this code, and because this code is designed as a synchronous set of lines, if we pass in an asynchronous `taskFunc` to `execute`, the events emitted will no longer be accurate. We can simulate the case with a `setImmediate` call:

```
// ...

withLog.execute(() => {
  setImmediate(() => {
    console.log('*** Executing task ***')
  });
});
```

The output we get for that will not be correct:

```
About to execute
Done with execute
*** Executing task ***
```

To emit an event after an asynchronous function is done, we'll need to combine callbacks (or promises) with the event-based communication. Here's an example:

```
import fs from 'node:fs';
import { EventEmitter } from 'node:events';
```

```

class WithTime extends EventEmitter {
  execute(asyncFunc, ...args) {
    this.emit('begin');
    console.time('execute');
    asyncFunc(...args, (err, data) => {
      if (err) {
        return this.emit('error', err);
      }

      this.emit('data', data);
      console.timeEnd('execute');
      this.emit('end');
    });
  }
}

const withTime = new WithTime();

withTime.on('begin', () => console.log('About to execute'));
withTime.on('end', () => console.log('Done with execute'));

withTime.execute(fs.readFile, import.meta.filename);

```

The `WithTime` class executes an `asyncFunc` and reports the time that's taken by that `asyncFunc` using `console.time` and `console.timeEnd` calls. It emits the right sequence of events before and after the execution.

We test a `withTime` emitter by passing it an `fs.readFile` call, which is an asynchronous function. Instead of handling file data with a callback, we can now listen to the data event.

When we execute this code, we get the right sequence of events, as expected, and we get a reported time for the execution:

```

About to execute
execute: 4.507ms
Done with execute

```

Note how we needed to combine a callback with an event emitter to accomplish that. If the `asyncFunc` supported promises as well, we could use the `async/await` feature to do the same:


```

class WithTime extends EventEmitter {
  async execute(asyncFunc, ...args) {
    this.emit('begin');
    try {
      console.time('execute');
      const data = await asyncFunc(...args);
      this.emit('data', data);
      console.timeEnd('execute');
      this.emit('end');
    } catch(err) {
      this.emit('error', err);
    }
  }
}

```

Errors

The `error` event is a special one. It should always be handled. Since we did not define a handler function for the `WithTime` `error` event, let's see what happens if we call the `execute` method with a bad argument. Add the bad call before the good one:

```

class WithTime extends EventEmitter {
  // ...
}

// ...

withTime.execute(fs.readFile, ''); // bad call

withTime.execute(fs.readFile, import.meta.filename);

```

The first `.execute` call will trigger an error. The Node process will crash and exit:

```

events.js:163
    throw er; // Unhandled 'error' event
    ^

Error: ENOENT: no such file or directory, open ''

```

The second `.execute` call will be affected by this crash and will potentially not get executed at all.

If we register a listener for the special `error` event, the behavior of the Node process will change. Let's do that:

```
// ...

withTime.on('error', (err) => {
  // Do something with err, for example log it somewhere
  console.log(err)
});

withTime.execute(fs.readFile, ''); // BAD CALL

withTime.execute(fs.readFile, import.meta.filename);
```

With the special `error` event handled, the first `.execute` call will not cause the Node process to exit. The other `.execute` call will finish normally:

```
{ Error: ENOENT: no such file or directory, open '' errno: -2, code: 'ENOENT' ...
  execute: 4.276ms
```

However, there is a reason the default behavior for unhandled errors is to make the Node process exit. You need to be careful when you define handlers for errors. It's OK to handle an expected error and make an exception for it to not cause the Node process to exit, but all other errors should make the process exit. Unknown errors lead to unknown states of the code.

Examples

Many of Node's built-in modules have objects in their APIs that are instances of the `EventEmitter` class. These objects emit certain events that we can handle in our code.

For example, the Node “Hello World” simple HTTP server example that we saw in [Chapter 1](#) can be written using events:

```
import { createServer } from 'node:http';

const server = createServer();

server.on('request', (req, res) => {
  res.writeHead(200, { 'Content-Type': 'text/plain' });
  res.end('Hello World');
});

server.on('listening', () => {
  console.log('Server is running...');
});

server.listen(3000, '127.0.0.1');
```

This is possible because the `server` object is an instance of the `http.Server` class, which extends the `EventEmitter` class under the hood, and both `request` and `listening` are predefined events in that class.

Even the `req` and `res` objects (emitted as data for the `request` event) are event emitters!

In [Chapter 6](#), we're going to learn about Node streams, which are also event emitter objects. Here is a simple example of how we can read the content of a file with a stream object:

```
import { createReadStream } from 'node:fs';

const readStream = createReadStream('input.txt', 'utf8');

readStream.on('data', (chunk) => {
  console.log('Read chunk:', chunk);
});

readStream.on('end', () => {
  console.log('Reading finished');
});
```

Note how with this example we're reading the file one chunk at a time, instead of all at once when we use the `readFile` function. This is a very

powerful concept in Node that'll enable us to work with big objects in a memory-efficient way. We'll learn all about that in [Chapter 6](#).

Other Node built-in modules that have event emitter objects include `node:child_process`, `node:net`, `node:dgram`, `node:os`, and `node:cluster`. We'll see examples of these throughout the book.

Another way to handle emitted `error` events is to define a handler for the `uncaughtException` event that's defined on the `process` object. This event is triggered for all unhandled errors. It's generally a bad practice to use it, unless you need to do something before letting the process exist as it should:

```
process.on('uncaughtException', (err) => {  
  // An error was unhandled. Process should exist.  
  
  // Do your thing  
  
  // But force exit the process after  
  process.exit(1);  
});
```

However, multiple `error` events might happen at the same time. This means the `uncaughtException` listener shown might be triggered multiple times, which is a problem as any code you write in that handler will be executed multiple times.

Event emitter objects have a `once` method that can be used instead of the `on` method. Handler functions registered with the `once` method will be invoked only one time even if their event is triggered multiple times. This is a better method to use with the `uncaughtException` event to make sure that any code you write in that handler is executed only once, as we know that the process is going to exit anyway.

If multiple handler functions are registered for the same event, the invocation of those functions will be in order. The first handler function that we register will be the first function to get invoked when the event is triggered:

```
withTime.on('data', (data) => {
  console.log(`Length: ${data.length}`);
});

withTime.on('data', (data) => {
  console.log(`Characters: ${data.toString().length}`);
});

withTime.execute(fs.readFile, import.meta.filename);
```

In this example, the `Length:` line will appear before the `Characters:` line, because that's the order in which we defined those listeners.

If you need to define a new listener but have that listener invoked first, you can use the `prependListener` method:

```
withTime.on('data', (data) => {
  console.log(`Length: ${data.length}`);
});

withTime.prependListener('data', (data) => {
  console.log(`Characters: ${data.toString().length}`);
});

withTime.execute(fs.readFile, import.meta.filename);
```

In this version, the `Characters:` line will be logged first.

Finally, if you need to remove a listener, you can use the `removeListener` method.

Summary

Node's event-driven approach is a simple yet powerful concept that allows for great flexibility and control over what to do before, during, and after asynchronous operations.

We can use callbacks, promises, or event emitters to work with asynchronous operations. Node's event loop manages asynchronous tasks using

event queues.

Node has an events module that we can use to define objects that support multiple events and multiple handlers for these events. Handler functions are always listening to the events they are attached to and get executed when these events are triggered. Events are triggered to indicate a condition is met.

In the next chapter, we'll discuss how to handle errors in Node and how to debug Node applications.